

From Source to Execution: A Statically-Typed Interpreted Language with Type Inference

by

Aye Khin Khin Hpone (Yolanda Lim), st125970

Applegate T. Tun Oo, st126690

Win Htut Naing, st126687

AT70.07 — Programming Languages and Compilers

Assignment 2

Instructor: Prof. Phan Minh Dung

Teaching Assistant: Akaradet Sinsamersuk

Asian Institute of Technology

School of Engineering and Technology

Thailand

April 2026

ABSTRACT

This report presents the design and implementation of a statically-typed programming language developed for a programming languages and compilers assignment. The language supports four primitive types — Integer, Float, Boolean, and String — with type inference, arithmetic and boolean expressions, assignment statements, if-then-else and while-loop control flow, function abstraction with value parameter passing, and a built-in `print()` function. The implementation follows a classical compiler pipeline consisting of lexical analysis, recursive-descent parsing, abstract syntax tree construction, static type checking, and tree-walking interpretation. A command-line runner and a desktop user interface are provided so that script files can be loaded, executed, and inspected. The report describes the language grammar, the type inference algorithm, the implementation structure, and the testing used to validate the system.

CONTENTS

ABSTRACT	ii
1 INTRODUCTION	1
1.1 Report Structure	1
2 LANGUAGE DESIGN AND GRAMMAR	3
2.1 Types	3
2.1.1 Integer	3
2.1.2 Float	3
2.1.3 Boolean	3
2.1.4 String	3
2.1.5 Void (internal only)	4
2.2 Static Typing	4
2.3 Token Specification	4
2.4 Context-Free Grammar	5
2.4.1 Program Structure	5
2.4.2 Arithmetic Expressions	5
2.4.3 Boolean Expressions	5
2.4.4 Statements	6
2.4.5 Function Definitions and Calls	6
2.5 Operator Precedence and Associativity	6
3 LEXICAL ANALYSIS AND SYMBOL TABLE	7
3.1 Token representation	7
3.2 Lexer implementation	8

3.3	Symbol table and semantic structure	10
3.4	Role of the symbol table in the pipeline	11
4	RECURSIVE-DESCENT PARSING AND THE ABSTRACT SYNTAX TREE	13
4.1	Abstract syntax tree representation	13
4.1.1	Expressions	14
4.1.2	Statements	14
4.2	Parser implementation	15
4.2.1	Statement parsing	15
4.2.2	Expression parsing and precedence	16
4.2.3	Helper functions and lookahead	17
4.3	AST printing	17
4.4	Parser and type checker boundary	18
5	TYPE INFERENCE ALGORITHM	19
5.1	Overview	19
5.2	Type Environment	19
5.3	Inference Rules	19
5.3.1	Literals	19
5.3.2	No Operator Overloading	20
5.3.3	Arithmetic Expressions	20
5.3.4	Boolean Expressions	21
5.3.5	Assignment Statement	21
5.3.6	If-Then-Else	21
5.3.7	While-Loop	21
5.3.8	Function Definition and Call	21
5.4	Type Error Reporting	22
5.5	Example Type Inference Traces	22

6	SYSTEM ARCHITECTURE AND IMPLEMENTATION	24
6.1	Compiler pipeline	24
6.1.1	Entry points and shared pipeline	25
6.2	Project structure	25
6.3	Technology stack	25
6.4	Type checker	26
6.4.1	Scope of the implemented type system	26
6.5	Interpreter	26
6.5.1	Scope of the implemented runtime	27
6.6	Graphical user interface	27
7	TESTING AND VERIFICATION	28
7.1	Test Cases	28
7.1.1	Lexer and Symbol Table	28
7.1.2	Parser and AST	29
7.1.3	Arithmetic Expressions	30
7.1.4	Boolean Expressions	30
7.1.5	Assignment Statements	30
7.1.6	If-Then-Else	30
7.1.7	While-Loop	30
7.1.8	Functions	30
7.1.9	Print	30
7.1.10	Unary Minus	31
7.2	Type Error Detection	31
7.3	Automated Test Suite	31
7.4	Error Handling	32
8	CONCLUSION	33

REFERENCES	35
APPENDIX A: SOURCE CODE AVAILABILITY	36
APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS	37

CHAPTER 1

INTRODUCTION

Programming languages define the formal contract between a programmer’s intent and machine execution. Building a language processor from source text through to evaluated output requires a coherent pipeline of components, each of which must correctly transform its input before passing it downstream. Lexical analysis identifies atomic tokens; parsing imposes hierarchical structure; static type checking validates semantic correctness before execution; and interpretation evaluates the validated program. When these stages are connected cleanly, errors are caught at the earliest possible point, and the overall system remains tractable to reason about, test, and extend.

This report presents the design and implementation of a small statically-typed interpreted language that realises this pipeline in full. The language supports four primitive types — Integer, Float, Boolean, and String — and provides arithmetic and equality expressions, assignment, conditional and iterative control flow, user-defined functions with value parameter passing, and a built-in `print()` statement. Types are inferred from program context rather than declared explicitly, combining the safety of static typing with the conciseness of a declaration-free syntax. The implementation spans three principal components developed in pipeline order: the lexer and symbol table, the recursive-descent parser and abstract syntax tree, and the type checker with interpreter. Each component was completed before the next was begun, since each stage consumes the structured output of its predecessor.

The implemented system can be invoked from the command line or through an interactive desktop interface, both of which surface the four pipeline stages — tokens, AST, inferred type table, and execution output — for inspection. An automated regression suite of 54 tests verifies correctness across all stages and language features.

1.1 Report Structure

Chapter 2 presents the language design, token specification, grammar, and operator precedence rules. Chapters 3 and 4 cover the front-end implementation in pipeline order: lexical analysis and the symbol table, then recursive-descent parsing and the AST. Chapter 5 explains the type

inference algorithm and the semantic rules used by the checker. Chapter 6 presents system architecture and implementation, including the end-to-end pipeline, project layout, and technology stack. Chapter 7 discusses testing and verification, and Chapter 8 concludes the report.

CHAPTER 2

LANGUAGE DESIGN AND GRAMMAR

2.1 Types

The language supports four primitive types. These types are sufficient to cover the assignment requirements while keeping the semantics simple and predictable.

2.1.1 *Integer*

The Integer type represents whole numbers such as 0, 7, and 42. Integer literals are written as one or more decimal digits without quotation marks or a decimal point.

2.1.2 *Float*

The Float type represents real-valued numbers written with a decimal point, such as 3.14 or 20.5. Float values are used in arithmetic expressions and also arise as the result of any division. Division always produces a Float even when both operands are Integer: for example, `10 / 2` evaluates to `5.0 : Float`. This is an explicit design rule enforced in the type checker.

2.1.3 *Boolean*

The Boolean type represents logical truth values. The two Boolean literals are `true` and `false`. Boolean values are used in conditions and as results of comparison expressions.

2.1.4 *String*

The String type represents sequences of characters enclosed in double quotes, for example `"hello"`. Strings can be assigned to variables and printed through the built-in `print()` function. The surrounding quotes are stripped by the parser when the `Literal` node is created, so the value stored internally is the bare character sequence without surrounding quotes. At run-time, the built-in `print()` function re-adds double quotes around string values, so `print("plc")` outputs `"plc" : String`.

2.1.5 Void (internal only)

Void is not a programmer-visible type. The language has exactly four primitive types: Integer, Float, Boolean, and String. Void exists solely as an internal sentinel inside the `LanguageType` enum so that the symbol table can record a well-defined return type for functions that contain no return statement. Programmers cannot write Void in source code.

2.2 Static Typing

The language uses static typing, which means type errors are detected before execution rather than during program execution. No explicit variable type declarations are required. Instead, the type checker infers types from literals, assignments, operations, function calls, and return statements. Once a variable type is established, later assignments must remain consistent with that type. For example, the following program is rejected at the type-checking stage before any execution occurs:

```
1 x = 5;
2 x = "hello";    (* TypeError: Cannot assign String to variable 'x' of
   type Integer *)
```

This contrasts with dynamically-typed languages, where the same program would run without error until (or unless) a type-dependent operation was attempted.

2.3 Token Specification

Table 2.1 summarises the major token classes used by the language.

Table 2.1. Token Specification

Token Category	Example Lexemes
INT_LITERAL	0, 10, 42
FLOAT_LITERAL	3.14, 20.5
BOOL_LITERAL	true, false
STRING_LITERAL	"hello"
IDENTIFIER	variable and function names such as x, add
Arithmetic operators	+, -, *, /
Comparison operators	==, !=
Assignment operator	=
Keywords	if, else, while, def, return, print
Delimiters	() { } , ;

2.4 Context-Free Grammar

The language grammar can be described by the context-free grammar $G = (V_N, V_T, P, S)$, where V_N is the set of non-terminals, V_T is the set of terminals, P is the set of production rules, and S is the start symbol program. The grammar is implemented through recursive-descent parsing functions.

2.4.1 Program Structure

```
1 program -> statement* EOF
2 statement -> assignment
3           | if_stmt
4           | while_stmt
5           | func_def
6           | print_stmt
7           | return_stmt
8           | expr ';' ;
```

2.4.2 Arithmetic Expressions

```
1 expr -> term (('+' | '-' ) term)*
2 term -> factor (('*' | '/' ) factor)*
3 factor -> '-' factor
4         | INT_LITERAL
5         | FLOAT_LITERAL
6         | STRING_LITERAL
7         | BOOL_LITERAL
8         | IDENTIFIER
9         | IDENTIFIER '(' args? ') '
10        | '(' expr ') ' ;
```

The unary minus production allows negative literals such as -5 and negated variable references such as -x. It is implemented as a right-recursive descent into factor and is desugared internally to `0 - factor`, so all existing type rules for arithmetic apply without modification.

2.4.3 Boolean Expressions

```
1 bool_expr -> expr '==' expr
2           | expr '!=' expr
3           | BOOL_LITERAL
4           | expr (* fallback: type checker enforces Boolean *)
```

Boolean expressions are only parsed as conditions inside if and while statements. Comparison results are not storable in variables directly; the grammar does not include `bool_expr` in the factor rule. This is consistent with the assignment specification, which describes Boolean

expressions primarily as conditions.

If `bool_expr` parses an expression and finds no following `==` or `!=` operator, the expression is returned as-is and the type checker verifies that its inferred type is `Boolean`.

2.4.4 Statements

```
1 assignment -> IDENTIFIER '=' expr ';'
2 if_stmt    -> 'if' '(' bool_expr ')' block ('else' block)?
3 while_stmt -> 'while' '(' bool_expr ')' block
4 print_stmt -> 'print' '(' expr ')' ';'
5 return_stmt -> 'return' expr ';'
6 block      -> '{' statement* '}'
```

2.4.5 Function Definitions and Calls

```
1 func_def -> 'def' IDENTIFIER '(' params? ')' block
2 params   -> IDENTIFIER (',' IDENTIFIER)*
3 args     -> expr (',' expr)*
```

Function calls use value parameter passing. This means the argument values are evaluated before the call and copied into the function's local environment.

2.5 Operator Precedence and Associativity

Operator precedence is encoded structurally in the recursive-descent parser. Comparison is lowest, addition and subtraction are next, and multiplication and division have the highest precedence among binary operators. Arithmetic operators are left-associative.

Table 2.2. Operator Precedence (1 = lowest, 3 = highest) and Associativity

Precedence	Operators	Associativity
1 (lowest)	<code>==</code> , <code>!=</code>	non-associative (condition use only)
2	<code>+</code> , <code>-</code>	left-associative
3 (highest)	<code>*</code> , <code>/</code>	left-associative

CHAPTER 3

LEXICAL ANALYSIS AND SYMBOL TABLE

This chapter covers the responsibilities related to lexical and early semantic foundations of the language. It includes defining token structures and categories (such as keywords, operators, identifiers, and literals), implementing the lexer to transform source characters into tokens, and establishing the symbol table to manage variable and function storage along with their associated types. Together, these components ensure that raw source input is systematically organised into meaningful structures that can be used reliably by later stages of the compiler.

3.1 Token representation

The token system, written in `components/tokens.py`, establishes the vocabulary and structure of all lexical elements in the language. It specifies both the types of tokens that can exist and the structure each token must follow. This serves as a canonical contract between the lexer and the parser, ensuring consistency across all stages of the compiler.

Table 3.1 summarises the main categories, example lexemes, and typical `TokenType` values.

Table 3.1. Token categories and representative `TokenType` values

Category	Example lexemes	Representative <code>TokenType</code>
Literals	42, 3.14, true, "hello"	INT_LITERAL, FLOAT_LITERAL, BOOL_LITERAL, STRING_LITERAL
Identifiers	x, counter, add	IDENTIFIER
Keywords	if, else, while, def, return, print	keyword-specific token variants
Operators	+, -, *, /, =, ==, !=	PLUS, MINUS, STAR, SLASH, ASSIGN, EQUAL_EQUAL, BANG_EQUAL
Delimiters	Parentheses, braces, comma, and semicolon (see lexer delimiter tokens)	delimiter token variants
End marker	end of source input	EOF

Token types (from class `TokenType`) are defined using an Enum with `auto()` to automatically assign unique values. This avoids reliance on raw strings and ensures safer comparisons during parsing. The token categories include literals, identifiers, keywords, operators, delimiters, and a special end-of-file (EOF) token. The EOF token is appended after all input is processed,

allowing the parser to detect the end of input explicitly rather than encountering an unexpected termination.

Each token is represented as an immutable dataclass (`frozen=True`), meaning its fields cannot be modified after creation. Each token contains four key pieces of information: its kind (`token_type`), its lexeme (the exact substring from the source code), and its source position (`line`, `column`). This structured representation allows later stages to process the program in terms of syntax and errors without handling raw text directly. Immutability improves reliability by preventing accidental modification during later processing.

```
1 @dataclass(frozen=True)
2 class Token:
3     token_type: TokenType
4     lexeme: str
5     line: int
6     column: int
```

3.2 Lexer implementation

The lexer, implemented in `components/lexica.py`, is a hand-written scanner that converts raw source text into a sequence of tokens. Unlike generated lexers (for example SLY lexer mode, or Lex and Flex), this implementation uses explicit control flow (`if` and `while`) to process input.

The lexer maintains internal state including the source string, current position, start of the current token, line number, and column index. The column index is tracked per line and resets upon encountering a newline, while a separate `token_column` records the starting position of each token for accurate error reporting.

The main function, `tokenize`, iterates through the source code, repeatedly invoking `_scan_token` to identify the next token. Tokens are appended to a list, and an EOF token is added at the end.

Whitespace is skipped, while invalid input results in a `LexerError`.

```
1 def tokenize(self) -> list[Token]:
2     tokens: list[Token] = []
3     while not self._is_at_end():
4         self.start = self.current
5         self.token_column = self.column
6         token = self._scan_token()
7         if token is not None:
8             tokens.append(token)
9     tokens.append(Token(TokenType.EOF, "", self.line, self.column))
10    return tokens
```

Token recognition follows a structured approach. Single-character tokens are mapped using a lookup table. Multi-character operators such as `==` and `!=` are handled using lookahead via `_match`. String literals are processed by `_string`, which reads characters until a closing quotation mark or the end of input is reached, with error handling for unterminated strings. Numeric literals are handled by `_number`, which distinguishes integers and floating-point values based on the presence of a decimal point followed by digits. Identifiers are processed greedily, consuming alphanumeric characters and underscores, and then checked against the `KEYWORDS` map to determine whether they are reserved words or user-defined names. Notably, `true` and `false` are also entries in this map, mapping to `TokenType.BOOL_LITERAL` rather than a keyword-specific type. This means boolean literals are recognised through the same keyword-fallback mechanism as reserved words, preventing them from being scanned as user-defined identifiers.

```
1 KEYWORDS: dict[str, TokenType] = {
2     "if": TokenType.IF,
3     "else": TokenType.ELSE,
4     "while": TokenType.WHILE,
5     # ...
6     "true": TokenType.BOOL_LITERAL,
7     "false": TokenType.BOOL_LITERAL,
8 }
```

```
1 SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
2     "+": TokenType.PLUS,
3     "-": TokenType.MINUS,
4     "*": TokenType.STAR,
5     "/": TokenType.SLASH,
6     # ...
7 }
```

Helper functions such as `_advance`, `_peek`, and `_peek_next` manage character consumption and lookahead, while `_is_at_end` determines when input has been fully processed. Error messages include precise line and column information, aiding debugging and diagnostics.

Table 3.2 lists the main functions and data structures in `components/lexica.py` and their roles.

Table 3.2. Key functions and data in `lexica.py`

Function / data	Responsibility	Output / effect
<code>tokenize()</code>	Iterates through source, scans tokens, appends EOF	ordered token list
<code>_scan_token()</code>	Dispatches recognition based on character and lookahead	next token or skip
<code>_number()</code>	Parses numeric sequences, distinguishes int vs float	numeric literal token
<code>_string()</code>	Parses quoted text, checks termination	string token or error
<code>_identifier()</code>	Parses identifiers, resolves keyword vs user-defined	keyword or identifier token
KEYWORDS map	Maps reserved words to token types	token type selection
SINGLE_CHAR_TOKENS map	Maps single-character symbols	token type selection
<code>_advance()</code> , <code>_peek()</code> , <code>_match()</code>	Character consumption and lookahead	scanner state update

3.3 Symbol table and semantic structure

The symbol table, defined in `components/symbol_table.py`, operates at a higher level than the lexer. While the lexer processes raw text into tokens, the symbol table manages semantic information such as variable and function names, their types, and their scope.

The symbol table is implemented as a scoped structure, where each scope maintains its own dictionary of symbols and optionally references a parent scope. This enables proper handling of nested blocks and variable shadowing. Symbols are introduced through definition functions and retrieved through lookup operations, which search recursively through parent scopes if needed.

```

1 def lookup(self, name: str) -> Symbol:
2     symbol = self._symbols.get(name)
3     if symbol is not None:
4         return symbol
5     if self.parent is not None:
6         return self.parent.lookup(name)
7     raise SymbolTableError(f"Undefined symbol: {name}")

```

The type system is represented using a `LanguageType` enumeration, which includes fundamental types such as Integer, Float, Boolean, String, and Void. Symbols are modelled using an inheritance hierarchy: the base `Symbol` class stores a name and type, while subclasses such as `VariableSymbol` and `FunctionSymbol` include additional attributes. Variables track initialization state, while functions store parameter lists and return types. Table 3.3 summarises the main building blocks.

Table 3.3. Symbol table components and roles

Component	Stored information	Role in semantic analysis
Scoped symbol table	Symbols in current and parent scope	Supports nesting and shadowing via recursive lookup
LanguageType enum	Integer, Float, Boolean, String, Void	Defines the type system
Symbol (base)	Name and declared or inferred type	Base abstraction for entries
VariableSymbol	Variable metadata (including initialization state)	Tracks correct usage before assignment
FunctionSymbol	Parameters and return type	Validates calls and return consistency
lookup(name)	Recursive scope search	Resolves identifiers or reports undefined
SymbolTableError	Semantic error details	Reports duplicates, undefined symbols, invalid operations

Errors related to semantic violations are handled via `SymbolTableError`. These include duplicate declarations within the same scope, references to undefined symbols, and invalid operations such as attempting to mark a non-variable symbol as initialized.

3.4 Role of the symbol table in the pipeline

The symbol table is constructed during the type-checking phase rather than during lexical analysis. This guideline is followed in the implementation. The type checker initialises a global symbol table and populates it while traversing the AST, including defining variables and functions, managing nested scopes, and validating type correctness.

```

1 def run_pipeline(source_code: str) -> PipelineResult:
2     tokens = Lexer(source_code).tokenize()
3     tree = Parser(tokens).parse()
4     ast_text = ASTPrinter().print(tree)
5     checked_scope = TypeChecker().check(tree)
6     outputs: list[str] = []
7     Interpreter(output_callback=outputs.append).execute(tree)
8     return PipelineResult(
9         tokens=tokens,
10        ast_text=ast_text,
11        checked_scope=checked_scope,
12        outputs=outputs,
13    )

```

Once type checking is complete, the populated symbol table is returned as part of the pipeline result (`checked_scope`). The checker creates child scopes for blocks and functions during analysis, but the displayed `checked_scope` table represents the global scope entries returned

by the pipeline. It can then be displayed or used for further analysis.

This separation of concerns keeps lexical analysis focused on tokenization, while semantic meaning is enforced during type checking. For user-defined functions, definitions are first registered in the global scope with placeholder parameter and return types. These are inferred from the first valid call and refined through body checking; subsequent calls must match the inferred parameter types.

CHAPTER 4

RECURSIVE-DESCENT PARSING AND THE ABSTRACT SYNTAX TREE

This chapter presents the implementation of the recursive-descent parser and the associated abstract syntax tree (AST) model. Starting from lexer-produced tokens, the parser constructs a hierarchical Program AST that represents expressions, statements, and control-flow structure in a form suitable for downstream semantic analysis and execution. Implemented syntax coverage includes arithmetic expressions, assignments, if-then-else, while, function definitions, function calls, return, and built-in `print(...)` statements. Equality operators (`==`, `!=`) are parsed in condition contexts (if/while) through the parser’s boolean-expression path.

4.1 Abstract syntax tree representation

The AST is defined as a collection of Python dataclasses representing the structural elements of the programming language. It serves as an intermediate representation between parsing and later stages such as type checking and interpretation.

A key distinction exists between tokens and AST nodes. Tokens are flat and describe individual lexical elements such as operators or identifiers. In contrast, the AST is hierarchical. For example, while a token may represent the symbol `+`, a `BinaryOp` node represents an entire addition operation, including its left and right operands as subtrees.

All AST nodes inherit from a common base class, allowing them to store positional metadata (line, column). This ensures that errors detected in later stages can still reference precise locations in the original source code.

```
1 @dataclass
2 class ASTNode:
3     """Base class for every AST node. Carries source location for
4         diagnostics."""
5     line: int = field(default=0, kw_only=True)
6     column: int = field(default=0, kw_only=True)
```

4.1.1 Expressions

Expressions represent computations or values within the program. A `Literal` node stores values known at parse time, including integers, floating-point numbers, booleans, and strings. String literals are stored without surrounding quotation marks, as these are removed during parsing.

```
1 if tok.token_type == TokenType.STRING_LITERAL:
2     self._advance()
3     return Literal(value=tok.lexeme[1:-1], line=tok.line, column=tok.
        column)
```

An `Identifier` node represents the use of a variable name within an expression, indicating evaluation rather than declaration. A `BinaryOp` node models arithmetic and comparison operations such as addition, subtraction, multiplication, division, and equality checks, storing the operator along with references to left and right operands. A `FunctionCall` node represents function invocation, consisting of the function name and a list of argument expressions.

```
1 @dataclass
2 class BinaryOp(ASTNode):
3     """An arithmetic or comparison binary operation.
4
5     op is one of: '+', '-', '*', '/', '==', '!=',
6     Example:  a + b    x == 0
7     """
8     op: str
9     left: ASTNode
10    right: ASTNode
```

4.1.2 Statements

Statements represent actions or control flow within the program. An `Assign` node models variable assignment, pairing a variable name with a value expression, while `Print` and `Return` nodes each store a single expression and represent output and function return operations respectively. A `Block` node groups multiple statements within braces, representing a scoped execution sequence.

Control flow is handled via `If` and `While` nodes. The `If` node stores a condition, a then-block, and an optional else-block, while `While` stores a loop condition and body block. Function definitions are represented by `FunctionDef` nodes, which include the function name, a list of parameter names, and a body block. Parameter types are intentionally not assigned at parse time, as type inference is handled later by the type checker. Finally, `Program` serves as the

AST root, containing all top-level statements.

4.2 Parser implementation

The parser consumes the list of tokens generated by the lexer and produces a single Program AST node. It is implemented as a hand-written recursive-descent parser based on the project grammar. This avoids parser generators and instead relies on explicit functions for each grammar rule.

The parser maintains internal state consisting of the token list and a position index (`self._pos`) indicating the current token. Tokens are accessed via helper methods such as `_peek` and `_advance`. Parsing errors are reported using a custom `ParseError` exception with line and column information consistent with lexer diagnostics.

```
1 def parse(self) -> Program:
2     """Parse the full token stream and return the root Program node."""
3     line, col = self._peek().line, self._peek().column
4     statements: list[ASTNode] = []
5     while not self._is_at_end():
6         statements.append(self._statement())
7     return Program(statements=statements, line=line, column=col)
```

In addition to the main `parse()` entry point, the parser's statement dispatcher (`_statement`) implements a deterministic decision order over token patterns: function definition, conditional, loop, return, print, assignment, then expression statement fallback. This design is important because it prevents ambiguity between syntactically similar forms. In particular, assignments are recognized using one-token lookahead (`IDENTIFIER` followed by `=`), while identifier-led expressions that are not assignments are parsed as expression statements and must still end with a semicolon. This gives the parser full coverage of both declarative-style and expression-driven top-level statements without requiring a parser generator.

4.2.1 Statement parsing

The main parsing loop repeatedly processes statements until EOF, with the statement type determined via dispatch on the current token. Function definitions, conditionals, loops, returns, and prints each have dedicated parsing methods. Assignments are detected using one-token lookahead (`IDENTIFIER` followed by `ASSIGN`), preventing misclassification of function calls as assignments. If no specific statement pattern matches, the parser falls back to an expression statement and enforces a terminating semicolon.

```

1 # assignment: IDENTIFIER "=" expr ";"
2 if (
3     tok.token_type == TokenType.IDENTIFIER
4     and self._peek_next().token_type == TokenType.ASSIGN
5 ):
6     return self._assignment()
7 # fallback: bare expression statement expr ";"
8 node = self._expr()
9 self._expect(TokenType.SEMICOLON, "Expected ';' after expression")

```

4.2.2 Expression parsing and precedence

Expressions are parsed in layered precedence levels: `_expr` handles `+` and `-`, `_term` handles `*` and `/`, and `_factor` handles literals, identifiers and function calls, and parenthesized expressions. Boolean conditions for `if` and `while` are parsed by `_bool_expr`, which recognises `==` and `!=` after parsing the left-hand arithmetic expression. As a result, equality parsing is condition-focused in the current grammar implementation rather than a general infix operator available in every expression position. Function calls are recognised when an identifier is followed by `(`, with argument lists parsed as comma-separated expressions. Parenthesized expressions are supported to override precedence. Unary negation (`-x`, `-5`) is also handled at the `_factor` level: a leading `-` token causes the parser to recursively parse the operand and return `BinaryOp(0 - operand)`, so the existing type rules for subtraction cover negation without any additional inference code.

```

1 def _expr(self) -> ASTNode: # +, -
2     ...
3 def _term(self) -> ASTNode: # *, /
4     ...
5 def _factor(self) -> ASTNode: # unary -, literals, identifiers/calls,
6     grouped expr
7     ...

```

Figure 4.1 shows the AST produced for the statement `z = x + y * 2;`. Because `_term` handles `*` before `_expr` handles `+`, the multiplication sub-tree is nested one level deeper, encoding the higher precedence of `*` over `+` without any explicit priority table at runtime.

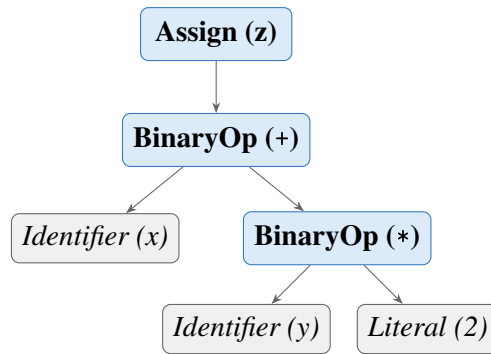


Figure 4.1. AST for $z = x + y * 2$; . Multiplication is a deeper sub-tree than addition, encoding higher precedence through tree structure rather than a runtime priority table.

Expression parsing is intentionally split into layered methods to enforce precedence and associativity in a transparent way. Parenthesized expressions are parsed explicitly and re-enter `_expr`, allowing precedence override where required by source syntax. Boolean conditions for `if` and `while` are parsed through `_bool_expr`, which accepts equality and inequality comparisons but can also return a non-comparison expression node; semantic enforcement that a condition is truly Boolean is deferred to the type checker.

4.2.3 *Helper functions and lookahead*

As described above, the parser uses `_check`, `_expect`, `_peek`, `_peek_next`, `_advance`, and `_is_at_end` for token navigation and validation. Lookahead is central for ambiguity resolution: assignment versus function call is resolved by inspecting the next token; comparison operators are recognised after parsing the left expression in `_bool_expr`. Parser errors follow a consistent diagnostic format: `[line X, col Y] ParseError: <message>`.

The parser also includes dedicated list parsers for formal parameters (`_params`) and call arguments (`_args`), both implemented as comma-separated sequences with optional emptiness. This ensures consistent handling of signatures such as `def f()` and `def f(a, b)` as well as invocations like `f()` and `f(x, y+1)`.

4.3 AST printing

For debugging and reporting, an AST printer converts the tree into a readable indented string, integrated into the pipeline output. The printer uses visitor-style dynamic dispatch, mapping node class names to methods such as `_visit_If` and `_visit_BinaryOp`, with output built as multi-line text where indentation reflects tree depth. Node formatting emphasises structure:

Program and Block include statement counts, BinaryOp explicitly shows left and right subtrees, and FunctionCall prints each argument by index. Literal includes both value and runtime Python type names for clarity.

4.4 Parser and type checker boundary

A deliberate architectural boundary is maintained between syntax construction and semantic validation. The parser is responsible for producing a structurally valid AST with accurate source locations, while type correctness and semantic constraints are checked later by the type checker. For example, `_bool_expr` may syntactically accept a non-Boolean expression as a condition node, but rejection of non-Boolean control-flow conditions is handled during semantic analysis. This separation keeps parser logic focused on grammar conformance and allows a clear responsibility split across team member contributions.

CHAPTER 5

TYPE INFERENCE ALGORITHM

5.1 Overview

Type inference in this project means that the programmer does not write explicit type declarations for variables or functions. Instead, the compiler infers types from assignments, literals, operations, function calls, and return statements. This keeps the language simple while still preserving static type safety.

The type checker runs after parsing. It receives the AST and validates the program before execution. Any inconsistent use of types is reported as a type error, and execution is stopped.

The interpreter stage does not repeat this validation: it walks the same AST with ordinary Python representations of values. Static type information remains in the symbol table produced by the checker; run-time values are not separately labelled with `LanguageType`, except that `print()` formats output by inspecting the value's Python type.

5.2 Type Environment

The type environment is represented by the symbol table. Conceptually, it is a mapping

$$\Gamma : \text{Identifier} \rightarrow \text{Type}$$

The environment grows as the checker visits the AST. On first assignment, a variable name is inserted with its inferred type. When a new block or a function call is entered, a child environment is created so that nested scopes can be checked independently while still accessing outer bindings when needed.

5.3 Inference Rules

5.3.1 Literals

Literal nodes have direct types:

$$\begin{aligned}\Gamma \vdash 42 &: \text{Integer} \\ \Gamma \vdash 3.14 &: \text{Float} \\ \Gamma \vdash \text{true} &: \text{Boolean} \\ \Gamma \vdash \text{"hi"} &: \text{String}\end{aligned}$$

5.3.2 No Operator Overloading

The language does not support operator overloading. Every operator has a single, fixed meaning determined solely by the operator symbol. Specifically:

- The `+` operator is not overloaded for string concatenation. Applying `+` to a `String` operand is a type error.
- The comparison operators `==` and `!=` are not overloaded for string or boolean equality. Both operands must be arithmetic.
- No mixed Boolean or String arithmetic is permitted.

Because operators have unambiguous types, the compiler can always infer expression types without requiring explicit type declarations from the programmer.

5.3.3 Arithmetic Expressions

The arithmetic operators `+`, `-`, `*`, and `/` accept only numeric operands. If both operands are `Integer`, the result is `Integer`, except for division, which always produces `Float` regardless of the operand types. If at least one operand is `Float` for `+`, `-`, or `*`, the result is `Float` — this is numeric promotion, not operator overloading, because `+` retains a single numeric meaning throughout. Any non-numeric operand causes a type error.

Table 5.1. Arithmetic result types by operand combination

Left operand	Operator	Right operand	Result type
Integer	<code>+</code> , <code>-</code> , <code>*</code>	Integer	Integer
Integer	<code>/</code>	Integer	Float
Float	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	Integer	Float
Integer	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	Float	Float
Float	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	Float	Float
Any	any	String or Boolean	TypeError

Unary negation

Unary minus ($-x$) is desugared by the parser into $0 - x$. The type checker therefore applies the standard binary subtraction rule: if x is `Integer`, the result is `Integer`; if x is `Float`, the result is `Float`.

5.3.4 Boolean Expressions

The comparison operators `==` and `!=` always produce `Boolean`. Following the language specification, both operators require two arithmetic (`Integer` or `Float`) operands. Comparisons involving `Boolean` or `String` operands are rejected by the type checker.

5.3.5 Assignment Statement

Assignments define variable types on first use. If a variable does not yet exist in the type environment, the checker infers the type of the right-hand side and inserts the variable into the environment. If the variable already exists, the new value must have the same type.

$$\text{If } x \notin \Gamma \text{ and } \Gamma \vdash e : T, \text{ then } \Gamma, x : T \vdash x = e$$
$$\text{If } x : T \in \Gamma \text{ and } \Gamma \vdash e : T, \text{ then } \Gamma \vdash x = e$$

5.3.6 If-Then-Else

The condition of an `if` statement must be `Boolean`. The `then`-block and `else`-block are checked in child scopes so that local operations can be validated cleanly.

$$\Gamma \vdash c : \text{Boolean} \quad \Gamma' \vdash B_{\text{then}} \quad \Gamma'' \vdash B_{\text{else}} \quad \Longrightarrow \quad \Gamma \vdash \text{if}(c) B_{\text{then}} \text{ else } B_{\text{else}}$$

5.3.7 While-Loop

The condition of a `while` loop must also be `Boolean`. The loop body is checked in a child scope and must not violate the types of variables already introduced in the surrounding environment.

$$\Gamma \vdash c : \text{Boolean} \quad \Gamma' \vdash B \quad \Longrightarrow \quad \Gamma \vdash \text{while}(c) B$$

5.3.8 Function Definition and Call

Function parameter types are inferred from the first call to the function. After that first call, subsequent calls must use arguments of the same types. The return type is inferred from the

return statements in the function body. If the function returns inconsistent types, the checker reports an error.

For functions that are defined but never called, the type checker performs a post-pass after all top-level statements have been processed. It infers parameter types by scanning the body for arithmetic and comparison usage, then checks the full body with those inferred types. This ensures that type errors inside never-called function bodies are still detected before execution.

Recursive calls are intentionally rejected in this implementation because they were not required by the assignment and would require additional inference and control-flow analysis.

5.4 Type Error Reporting

Type errors are reported with source positions so that the user can locate the problem quickly. The format used by the system is:

```
[line X, col Y] TypeError: message
```

Examples include assigning a `String` to an `Integer` variable, using a non-Boolean condition in an `if` statement, or supplying the wrong argument type to a function.

5.5 Example Type Inference Traces

Consider the following program fragment:

```
1 x = 10;  
2 y = 20.5;  
3 z = x + y;  
4 print(z);
```

The checker infers:

- `x`: `Integer`
- `y`: `Float`
- `x + y`: `Float`
- `z`: `Float`

For a function example:

```
1 def add(a, b) {  
2     return a + b;  
3 }  
4  
5 result = add(2, 3.5);
```

The first function call infers `a`: `Integer` and `b`: `Float`. The expression `a + b` is therefore `Float`, so the function return type is inferred as `Float`. The variable `result` is also inferred as `Float`.

CHAPTER 6

SYSTEM ARCHITECTURE AND IMPLEMENTATION

This chapter describes how the implementation is organised: the end-to-end processing flow, repository layout, technology choices, and the main components that realise static analysis and execution (type checker, interpreter, and user interfaces). Detailed treatment of lexical analysis, the symbol table, and parsing appears in Chapters 3 and 4; Chapter 5 presents the type-inference rules that the type checker implements.

6.1 Compiler pipeline

The system follows a four-stage pipeline. Lexical and syntactic processing are kept separate from semantic analysis and execution, which simplifies testing and localises errors to the appropriate stage.

The overall flow of data through the system is shown in Figure 6.1.

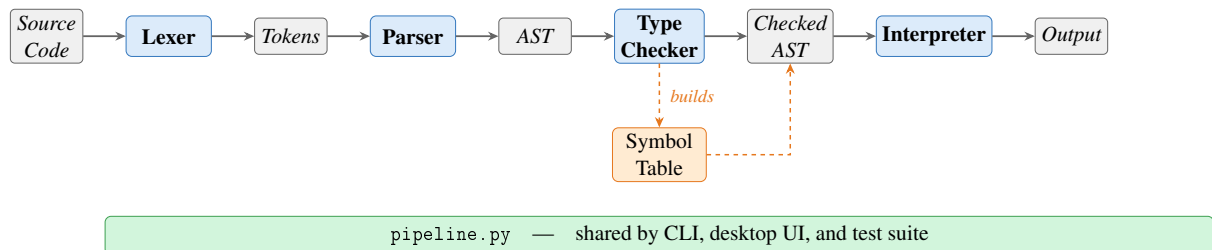


Figure 6.1. Compiler pipeline overview.

Source characters are scanned by the lexer into a token stream. The parser consumes tokens and builds an AST that reflects program structure (expressions, statements, and control flow). The type checker then walks the AST, populates and uses the symbol table, infers types, and validates scope and type rules. Only programs that pass the type checker proceed to interpretation. The interpreter evaluates the checked tree using ordinary Python values; it does not re-run the type rules, since correctness at runtime is guaranteed by the prior static pass. All four stages are orchestrated through `pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite.

6.1.1 Entry points and shared pipeline

All stages are orchestrated by `components/pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite. From the project root, `python3 main.py` runs a script (or built-in sample) and prints the stages in order: tokens, AST text, type table, and execution output. `python3 ui.py` runs the Tkinter workbench: the same pipeline is used internally, with each stage shown in a separate tab so the user can inspect results without altering the underlying flow.

6.2 Project structure

The main source files and their responsibilities are summarised in Table 6.1.

Table 6.1. Main source files

File	Responsibility
<code>main.py</code>	Command-line runner for sample code or a script file
<code>ui.py</code>	Desktop UI for editing and running a script
<code>components/tokens.py</code>	Token classes and token categories
<code>components/lexica.py</code>	Lexical analysis
<code>components/ast_nodes.py</code>	AST node definitions
<code>components/parser.py</code>	Recursive-descent parser
<code>components/ast_printer.py</code>	AST renderer for debugging and reporting
<code>components/symbol_table.py</code>	Scoped symbol table and semantic types
<code>components/type_checker.py</code>	Static type checker and inference
<code>components/interpreter.py</code>	Tree-walking interpreter
<code>components/pipeline.py</code>	Shared pipeline for CLI, UI, and tests

6.3 Technology stack

The project uses Python and standard-library tools only (Table 6.2).

Table 6.2. Technology stack

Tool	Purpose
Python 3.10+	Core implementation language
<code>dataclasses</code>	AST and runtime helper structures
<code>tkinter</code>	Desktop graphical interface
<code>unittest</code>	Automated testing

6.4 Type checker

The `TypeChecker` class implements the inference and checking rules described in Chapter 5. It traverses the AST before execution, uses the symbol table to record variable and function information, infers types on first assignment, validates arithmetic and comparison expressions, enforces Boolean conditions for control flow, infers function parameter types from the first call (or from body usage for uncalled functions), and infers function return types from `return` statements. Programs that fail these rules are rejected before interpretation.

6.4.1 Scope of the implemented type system

In scope terms, this component turns a syntactically valid AST into a semantically valid program:

- static typing rules for arithmetic, comparison, assignment, control flow, and functions,
- type checking for expressions and statements,
- variable type inference from first assignment,
- function parameter inference from the first call (or from body usage for uncalled functions),
- function return type inference from `return` statements,
- source-positioned type errors when a program violates a typing rule.

6.5 Interpreter

The interpreter is a tree-walking evaluator over the checked AST. It maintains nested runtime environments for variables and function calls. Assignments write through the current or nearest enclosing scope; `if` selects a branch; `while` repeats until its condition becomes false; function calls bind arguments by value and return through an internal return mechanism.

The built-in `print()` function outputs both value and runtime type. All four types are supported:

```
1 print(42);           (* 42 : Integer      *)
2 print(3.14);         (* 3.14 : Float      *)
3 print(true);         (* true : Boolean    *)
4 print("plc");        (* "plc" : String   *)
```


6.5.1 *Scope of the implemented runtime*

The runtime stage provides observable behaviour and integrates with the shared pipeline:

- assignment execution,
- if execution,
- while execution,
- function execution with value parameter passing,
- `print()` execution with immediate output,
- execution behind the same `run_pipeline` path used by the CLI, UI, and tests.

6.6 Graphical user interface

The workbench is implemented with `tkinter`. The layout required for the assignment is preserved: the script is edited on the left, and pipeline outputs appear on the right. The right-hand side surfaces the same stages as the command-line printout, but in a tabbed panel so the user can switch between views without rerunning the pipeline for each kind of inspection.

CHAPTER 7

TESTING AND VERIFICATION

7.1 Test Cases

Testing was performed through both manual execution and automated regression tests. Manual runs were used to inspect the full pipeline from source code to output, while automated tests were added for the semantic and runtime behaviour of the implemented type checker and interpreter.

Table 7.1. Representative Test Cases

Category	Program Behaviour	Expected Result
Arithmetic	Evaluate mixed numeric expressions	Correct value and inferred type
If-then-else	Choose one branch from a Boolean condition	Only matching branch runs
While-loop	Print during repeated execution	One output line per iteration
Functions	Infer parameters and return type	Valid call and correct return value
Type mismatch	Reassign incompatible type	Type error before runtime

7.1.1 *Lexer and Symbol Table*

The lexer was verified manually by running the full pipeline and inspecting Stage 1 token output for programs that exercise every token category: integer and float literals, Boolean literals, string literals, all six keywords, two-character comparison operators (`==`, `!=`), single-character operators, and delimiters. Identifier scanning with keyword fallback was confirmed by checking that `true`, `false`, and reserved words are emitted with their keyword token types while non-keyword names produce `IDENTIFIER` tokens. For example, the source fragment `x = 10;` produces the token sequence:

```

1 IDENTIFIER  'x'      line=1 col=1
2 ASSIGN      '='      line=1 col=3
3 INT_LITERAL '10'     line=1 col=5
4 SEMICOLON   ';'      line=1 col=7

```

Error paths were also checked: supplying a bare `!` character raises a `LexerError` with a source position and a message suggesting `!=`, and an unterminated string literal raises a `LexerError` citing the opening line.

The symbol table was verified through the type checker and the Stage 3 output. Duplicate variable definitions were confirmed to raise a `SymbolTableError` before execution. Scope isolation was verified with nested blocks: a variable defined inside an `if` or `while` body is not visible in the outer scope after the block ends. The `format_table()` output was checked against the expected column layout (name, kind, type, initialisation status, parameters) for both variable and function symbols.

7.1.2 *Parser and AST*

The parser was verified manually by running the full pipeline and inspecting Stage 2 AST output. The following cases were checked:

- arithmetic expressions with mixed precedence (e.g. `x + y * 2`) produce a tree where multiplication is a deeper sub-tree than addition, confirming the `_expr/_term/_factor` precedence encoding,
- assignment versus expression statement dispatch: `x = 5`; produces an `Assign` node while `add(1,2)`; produces a `FunctionCall` node at the statement level, confirming the `IDENTIFIER+ASSIGN` lookahead,
- `if` with and without an `else` clause: the resulting `If` node has a non-null `else_block` only when the clause is present,
- function definitions store only parameter names in the `FunctionDef` node; the AST printer output was inspected to confirm no type annotations are present at this stage,
- string literals in `Literal` nodes have their surrounding double-quotes stripped; `print("hello")` prints `"hello" : String` at runtime (the interpreter re-adds the quotes when formatting string output),
- parse errors: missing semicolons, unmatched parentheses, and unexpected tokens all produce `[line X, col Y] ParseError`: messages with correct source positions.

7.1.3 Arithmetic Expressions

Arithmetic expressions were tested using both integer-only and mixed integer-float programs. These tests verified precedence and numeric promotion. Expressions such as `x + y * 2` confirmed that multiplication is applied before addition.

7.1.4 Boolean Expressions

Boolean expressions were tested inside `if` and `while` conditions. The type checker enforces that both operands of `==` and `!=` are arithmetic (Integer or Float); supplying a Boolean or String operand raises a `TypeCheckError`. The interpreter used the resulting Boolean values for control flow.

7.1.5 Assignment Statements

Assignments were tested for both first-use inference and reassignment checks. For example, a variable assigned 5 and later assigned "hello" correctly triggers a type error.

7.1.6 If-Then-Else

Conditional programs were used to verify that exactly one branch is executed and that non-Boolean conditions are rejected before runtime.

7.1.7 While-Loop

While-loops were tested with a countdown example. This confirmed that the loop body executes repeatedly and that `print()` emits output progressively on each iteration rather than only once at the end.

7.1.8 Functions

Function tests covered parameter passing, local execution, and return values. The implementation infers parameter types from the first call and then enforces that signature for later calls.

7.1.9 Print

The built-in `print()` function was tested with all supported runtime types. The output includes the printed value together with its type, such as `29.5 : Float` and `"plc" : String`.

7.1.10 Unary Minus

The unary minus operator was verified for integer literals ($x = -5$), float literals ($y = -3.14$), negated variable references ($y = -x$), and negation inside a larger expression ($x = 3 + -2$). In each case the type checker assigned the correct type and the interpreter produced the expected value.

7.2 Type Error Detection

The checker was verified with invalid programs involving incompatible assignment, invalid arithmetic operands, wrong function arguments, and non-Boolean conditions. In each case the system stopped before execution and reported a source-positioned type error. Representative error messages are shown below.

```
1 # Assignment type mismatch
2 [line 2, col 1] TypeError: Cannot assign String to variable 'x' of type
   Integer
3
4 # Non-Boolean if condition
5 [line 1, col 4] TypeError: If condition must have type Boolean
6
7 # Wrong argument type for function
8 [line 5, col 9] TypeError: Argument 1 of 'add' must be Integer, got
   String
```

7.3 Automated Test Suite

The automated regression suite is located in `tests/test_pipeline.py` and uses Python's built-in `unittest` framework. The tests are organised into five classes, each targeting a distinct compiler stage.

Table 7.2. Automated test coverage by class

Test class	Stage covered	Tests
LexerTests	Tokenisation (<code>lexica.py</code>)	10
SymbolTableTests	Scoped symbol table (<code>symbol_table.py</code>)	7
ParserTests	Parsing and precedence (<code>parser.py</code>)	5
TypeCheckerTests	Static type inference (<code>type_checker.py</code>)	15
IntegrationTests	Full pipeline execution	17
Total		54

LexerTests verify that each token category (integer, float, Boolean, string, keywords, identi-

fiers, two-character operators) is produced correctly, that source line numbers are tracked across newlines, and that illegal characters and unterminated strings raise `LexerError`.

SymbolTableTests exercise the symbol table directly, without going through the full pipeline. They cover variable definition and lookup, duplicate definition raising `SymbolTableError` for both variables and functions, parent-scope lookup through `child_scope()`, undefined-symbol lookup error, and the `format_table()` output for both variable and function symbol rows.

TypeCheckerTests verify static type inference and error detection. The 15 tests cover integer and float arithmetic, division producing `Float`, numeric promotion, string and boolean type inference, assignment type mismatch, non-Boolean conditions, undefined variables, wrong argument count and type, and two tests for uncalled-function body checking: one confirms that a type error inside a never-called function body is detected, and another confirms that a valid uncalled function raises no error.

IntegrationTests run programs through all four pipeline stages and verify the final output. Cases include unary negation on all numeric types, both branches of `if/else`, while-loop countdown, `print()` with every type, value-parameter isolation, nested function calls, variable reassignment, and scope isolation (variables declared inside an `if` block are not visible outside it).

The tests can be run with:

```
python3 -m unittest discover -s tests -v
```

All 54 tests pass.

7.4 Error Handling

The language system reports errors at three main stages:

- lexical errors for malformed characters or unterminated strings,
- parse errors for invalid syntax such as missing semicolons,
- type errors for programs that violate static typing rules.

Runtime errors are also reported with source positions whenever execution reaches an invalid state such as an undefined variable reference.

CHAPTER 8

CONCLUSION

This project delivered a working statically-typed programming language with the required primitive types: Integer, Float, Boolean, and String. The language supports arithmetic expressions (including unary negation), equality and inequality checks, assignments, if-then-else statements, while-loops, function abstraction with value parameter passing, return statements, and the built-in `print()` function.

The final system follows a clean compiler-style architecture. The lexer creates tokens, the parser builds the AST, the type checker performs static inference and validation, and the interpreter executes the program. This staged design made it easier to isolate syntax errors, type errors, and runtime behaviour.

One of the main outcomes of the project is the successful implementation of static typing without explicit declarations. Variable types are inferred from assignments, and function signatures are inferred from calls and return statements. Invalid programs are rejected before runtime. In addition, the system includes both a command-line runner and a desktop UI so that scripts can be entered, loaded, executed, and inspected conveniently.

The 54 automated tests cover all five test classes — lexical analysis, symbol table, parsing, static type checking, and full pipeline execution — and confirm correct behaviour for all language features including scope isolation, duplicate symbol detection, unary negation, division type rules, and error detection. These checks provide a baseline for future extensions.

One design consideration in the type checker is that function bodies are checked on demand. For functions that are called, the body is checked at the first call site using the inferred argument types. For functions that are defined but never called, the type checker performs a post-pass after all top-level statements have been processed: it infers parameter types from how they are used inside the body (for example, a parameter used in an arithmetic expression is inferred as Integer or Float), then runs the full body check with those inferred types. This ensures that type errors in never-called function bodies are still detected before execution.

Future work could include recursive functions, richer data structures, additional operators, and

code generation to a lower-level target. The current implementation demonstrates all core compiler-construction concepts covered in the course — lexical analysis, parsing, static type inference, and tree-walking interpretation — in a complete, tested, and working system.

REFERENCES

- Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, 2nd edition, 2006.
- Robert Nystrom. *Crafting Interpreters*. Genever Benning, Seattle, WA, 2021.
- Python Software Foundation. Python documentation: dataclasses — data classes. <https://docs.python.org/3/library/dataclasses.html>, 2026a. Accessed: April 2026.
- Python Software Foundation. Python documentation: Tkinter — python interface to tcl/tk. <https://docs.python.org/3/library/tkinter.html>, 2026b. Accessed: April 2026.
- Python Software Foundation. Python documentation: unittest — unit testing framework. <https://docs.python.org/3/library/unittest.html>, 2026c. Accessed: April 2026.

APPENDIX A: SOURCE CODE AVAILABILITY

The complete source code for this project is available on GitHub:

<https://github.com/The-Token-Trio/125970-126690-126687-Assignment2-PLC>

The repository contains the lexer, parser, AST nodes, symbol table, type checker, interpreter, shared execution pipeline, user interface, tests, and the full report source. The project can be run either from the command line through `main.py` or from the desktop UI through `ui.py`.

The main execution commands are:

```
1 python3 main.py
2 python3 ui.py
3 python3 -m unittest discover -s tests -v
```

No external Python dependencies are required for the implementation. The Python side of the project uses only the standard library.

APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS

Name	Student ID	Assignment Scope
Aye Khin Khin Hpone (Yolanda Lim)	st125970	Static typing rules; type checking; assignment execution; if execution; while execution; function execution; <code>print()</code> execution; unary minus type inference and execution; pipeline integration (<code>pipeline.py</code>); automated test suite (<code>test_pipeline.py</code> , 54 tests)
Applegate T. Tun Oo	st126690	Lexer implementation (character scanning, tokenisation, line/column tracking, error reporting); token definitions; keywords and operators; identifiers and literals; variable/function storage; type storage
Win Htut Naing	st126687	Arithmetic expressions; boolean expressions; assignment statements; if-then-else; while-loop; function definitions; function calls; <code>print()</code> syntax; unary minus parsing